**[TÊN TRƯỜNG]**

**[KHOA / BỘ MÔN]**

# BÁO CÁO THỰC HÀNH

## MÔN: TRIỂN KHAI VÀ QUẢN TRỊ HỆ THỐNG PHẦN MỀM

## ĐỀ 22: HỆ THỐNG ĐẶT XE / TAXI BOOKING

**Sinh viên: Lý Minh Phương**

**Mã số sinh viên: Dtc245200323**

**Lớp: [LỚP]**

**Giảng viên giảng dạy: Nguyễn Anh Chuyên**

**Tháng 10 năm 2026**

# MỤC LỤC VÀ THÔNG TIN BẢN BÁO CÁO

Báo cáo thực hành môn Triển khai và Quản trị Hệ thống Phần mềm, đề 22: Hệ thống Đặt xe / Taxi Booking. Nội dung trình bày ứng dụng ba vai trò và hạ tầng triển khai bằng Docker Compose, kèm bằng chứng kiểm tra thực tế.

Các phần: (1) Mục tiêu và phạm vi; (2) Kiến trúc; (3) Database và pgAdmin; (4) Chức năng và phân quyền; (5) Triển khai; (6) Nginx; (7) Prometheus; (8) Grafana; (9) Loki + Promtail và LogQL; (10) Hardening; (11) Kiểm thử; (12) GitHub; (13) Demo và kết luận.

Trạng thái tài liệu: bản hoàn chỉnh về kỹ thuật, chờ bổ sung thông tin sinh viên và URL GitHub vào docs/student.json. Không khẳng định đã tạo tài khoản hoặc đã push GitHub khi chưa có thông tin và xác thực của sinh viên.

Thời điểm kiểm chứng hạ tầng: 2026-10-07T08:01:38.513683+00:00. File JSON minh chứng nằm trong docs/evidence; ảnh giao diện thật nằm trong docs/screenshots. Các ảnh thử nghiệm bản đồ có hậu tố map-test/motorbike-promotion-test dùng tile giả trong kiểm thử tự động, không dùng để chứng minh tile bản đồ thật trong báo cáo.

Định dạng: khổ A4, Times New Roman, cỡ chữ 12; bìa riêng, tiêu đề từng mục, số trang và hình có chú thích. Bản PDF có số trang được kiểm tra tự động; bản Word dùng cùng nội dung và ngắt trang theo mục.

# 1. MỤC TIÊU VÀ PHẠM VI ĐỀ TÀI

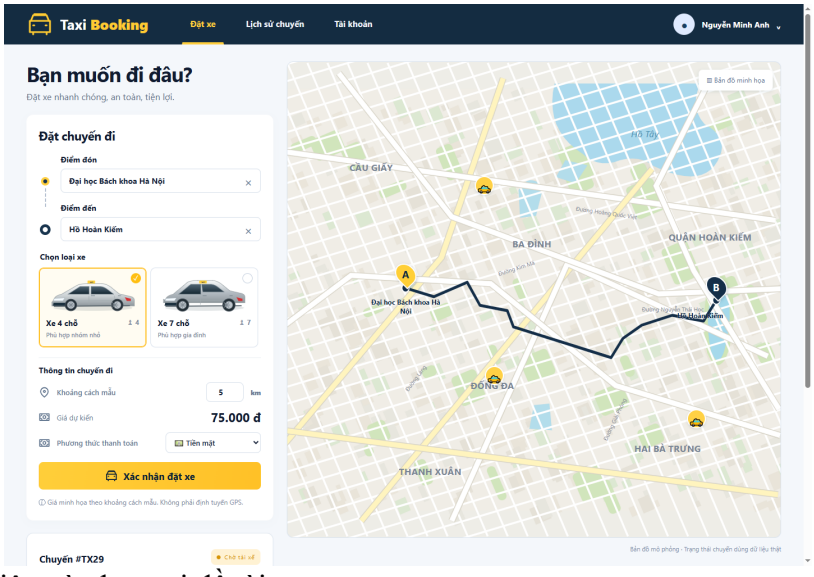
Bài thực hành yêu cầu một ứng dụng web kết nối database và công cụ quản trị, chạy hoàn toàn bằng Docker Compose. Hệ thống cần reverse proxy Nginx, giám sát Prometheus + Grafana cho web/database/container, log tập trung Loki + Promtail và các biện pháp bảo mật có minh chứng.

Đề 22 chọn nghiệp vụ đặt xe: khách đặt chuyến, tài xế nhận và thực hiện chuyến, quản trị viên quản lý toàn hệ thống. Ứng dụng sử dụng Flask/Gunicorn, HTML/CSS/JavaScript và PostgreSQL 16. Đây là lựa chọn công nghệ triển khai của dự án; vẫn đáp ứng database PostgreSQL + pgAdmin trong đề.

Phạm vi chức năng gồm xe máy một khách, ô tô 4/7 chỗ, bản đồ chọn vị trí, giá theo quãng đường, mã giảm giá, thanh toán tiền mặt/chuyển khoản QR và quản lý ảnh tài xế. Cước và giảm giá được tính tại máy chủ, không tin số tiền do trình duyệt gửi lên.

Thanh toán chuyển khoản được quản trị viên đối soát thủ công trước khi đánh dấu thành công hoặc thất bại. Không có webhook ngân hàng xác thực tự động. Tuyến đường dùng OSRM ô tô làm tham chiếu, cả khi chọn xe máy; chưa có định tuyến riêng xe máy hay dữ liệu giao thông trực tiếp.

Các dịch vụ công khai của bản đồ phục vụ demo đồ án với lưu lượng thấp. Triển khai lớn cần nhà cung cấp phù hợp. Mục tiêu của báo cáo là trình bày đúng phần đã triển khai, cách kiểm chứng và các giới hạn còn lại, không thay bằng kết quả giả lập.



Hình 2. Mục tiêu và phạm vi đề tài.

# 2. KIẾN TRÚC VÀ LUỒNG HOẠT ĐỘNG

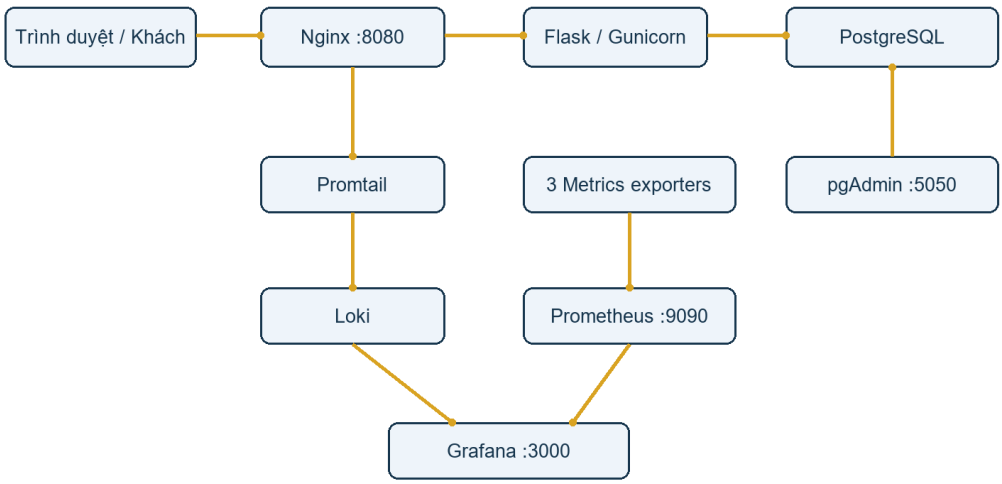
Trình duyệt truy cập cổng 8080 trên localhost; Nginx chuyển tiếp yêu cầu tới Gunicorn cổng 8000. Flask xác thực phiên, kiểm tra vai trò/CSRF và thực hiện truy vấn có tham số qua psycopg. PostgreSQL lưu dữ liệu trên named volume, không công bố cổng 5432 ra host.

Container migrate chạy một lần sau khi database healthy, thực hiện migration và tạo các role DB. App chỉ khởi động sau khi migrate kết thúc thành công. Tách bước này giúp tiến trình web không giữ mật khẩu superuser database; thay đổi mật khẩu tài khoản demo trong môi trường không ghi đè tài khoản đã tồn tại.

Mạng edge phục vụ dịch vụ cần giao diện localhost và truy cập nhà cung cấp bản đồ. backend là mạng internal cho DB/app/pgAdmin/exporter database. monitoring là mạng internal cho metrics và log. App nối nhiều mạng vì cần cả database và scrape API; DB chỉ nằm trong backend.

Metrics exporter Nginx đọc stub_status nội bộ; postgres-exporter đọc thống kê bằng role taxi_monitor; container-exporter đọc Docker Engine stats rồi xuất counters/gauges. Prometheus scrape mỗi 15 giây. Grafana hiển thị dashboard và truy vấn Loki.

Nginx ghi access/error log vào volume nginx_logs. Promtail đọc volume ở chế độ chỉ đọc, thêm nhãn job và collector rồi gửi Loki. Grafana dùng datasource Loki để truy vấn LogQL. Alloy được giữ trong profile tùy chọn, không chạy đồng thời với Promtail trong demo mặc định.




Hình 3. Kiến trúc và luồng hoạt động.

# 3. DATABASE VÀ CÔNG CỤ QUẢN TRỊ

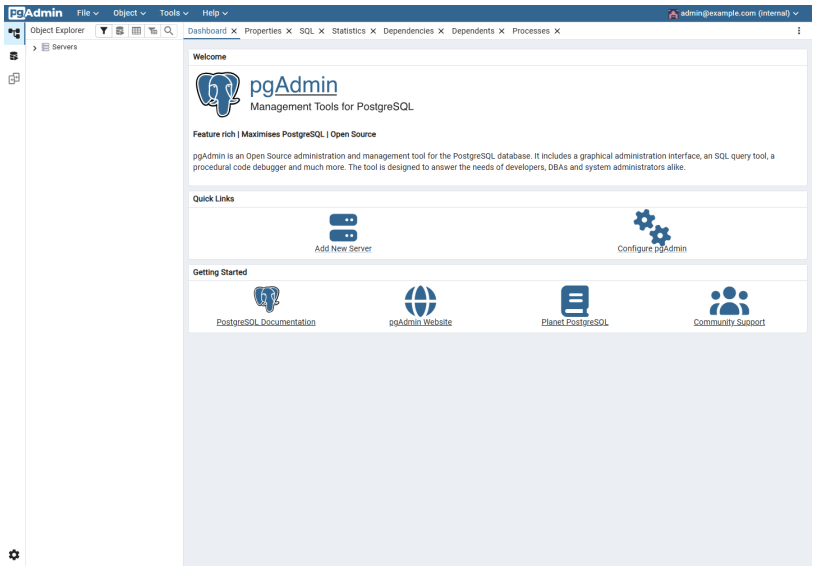
Các bảng chính: customers, drivers, users, sessions, vehicles, fares và trips. users liên kết đúng hồ sơ khách/tài xế; role admin không cần hồ sơ. sessions lưu hash token cùng CSRF token và hạn dùng, không lưu cookie token rõ trong database.

trips chứa điểm đón/đến, loại xe, tài xế, khoảng cách, cước, trạng thái và thời điểm. route_quotes lưu tuyến có hạn 15 phút theo người dùng. Vị trí GPS gần nhất của tài xế được lưu riêng trong driver_locations; khách chỉ được xem vị trí tài xế của chuyến đang phục vụ mình.

Biểu giá được chụp lại trong chuyến bằng base_fare/per_km để thay đổi giá mới không làm sửa lịch sử. promotions lưu kiểu giảm, mức tối đa, hạn dùng và giới hạn lượt. Chuyến lưu promo_code/discount_amount; payment_attempts giữ số tiền và tài khoản nhận của từng yêu cầu.

Khóa ngoại giữ toàn vẹn giữa khách, tài xế và chuyến; CHECK giới hạn trạng thái và loại xe 1/4/7. Unique partial index bảo đảm một tài xế chỉ có một chuyến assigned/ongoing. Nhận chuyến và dùng mã giảm giá có khóa hàng trong transaction để xử lý tranh chấp đồng thời.

pgAdmin truy cập tại http://127.0.0.1:5050. Đăng nhập bằng PGADMIN_EMAIL/PGADMIN_PASSWORD trong .env. Đăng ký server: Host db, Port 5432, Database taxi, User taxi và mật khẩu POSTGRES_PASSWORD. Dùng Query Tool với SELECT id,pickup,destination,fare,status FROM trips ORDER BY id DESC LIMIT 20 để trình bày dữ liệu. Không đưa mật khẩu vào ảnh hoặc repository.



Hình 4. Database và công cụ quản trị.

# 4. CHỨC NĂNG VÀ PHÂN QUYỀN

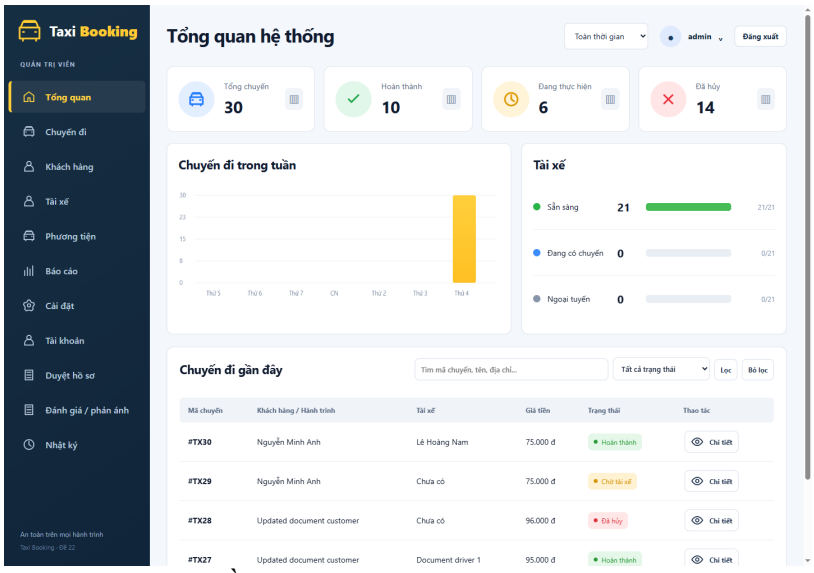
Khách hàng đăng ký/đăng nhập, tìm hoặc chọn điểm trên bản đồ, chọn xe máy/4/7 chỗ, áp dụng mã giảm giá và xác nhận đặt chuyến. Có lịch sử riêng, hủy khi đang chờ nhận, theo dõi GPS của tài xế đang phục vụ, đánh giá và gửi phản ánh.

Tài xế bật/tắt nhận chuyến và chỉ thấy hàng chờ đúng phương tiện đang được duyệt. Sau nhận chuyến, tài xế báo đã đến, bắt đầu và hoàn thành. Hồ sơ, giấy phép, phương tiện, ảnh từ thư viện và thu nhập được quản lý riêng. GPS chỉ bật khi tài xế chủ động cho phép và có nút dừng.

Admin xem tất cả chuyến, quản lý khách/tài xế/tài khoản/phương tiện, khóa tài khoản, reset mật khẩu, duyệt yêu cầu phương tiện, chỉnh biểu giá, xem báo cáo/CSV và audit. Admin tạo quảng cáo mã giảm giá, bật/tắt mã, kiểm tra chuyển khoản thành công/thất bại có ghi chú.

Backend kiểm tra quyền ở từng API; ẩn nút trên giao diện không được xem là bảo mật. Khách không thể nhận chuyến hay truy cập khách khác; tài xế không lấy thông tin liên hệ của chuyến chưa nhận. Admin có nhật ký hành động để giải thích thay đổi nghiệp vụ.

Thông tin liên hệ và lịch sử thanh toán được giới hạn theo chủ sở hữu. GPS cũ trên 120 giây không hiển thị như vị trí hiện tại. Đăng xuất/dừng chia sẻ/hoàn thành chuyến làm ngừng hiển thị vị trí. Cước từ route_quote và giảm giá được máy chủ kiểm tra lại trong cùng giao dịch đặt chuyến.



Hình 5. Chức năng và phân quyền.

# 5. TRIỂN KHAI BẰNG DOCKER COMPOSE

Điều kiện: Docker Desktop chạy Linux containers, Docker Compose và Git. Clone repository vào thư mục làm việc, sao chép .env.example thành .env nếu chưa có, thay mật khẩu mẫu. File .env nằm trong .gitignore và .dockerignore, không được commit hoặc đóng gói vào image.

Lệnh triển khai: docker compose config --quiet để kiểm tra cấu hình, tiếp theo docker compose up -d --build. Dùng docker compose ps -a để xem cả container migrate kết thúc thành công. App và PostgreSQL có healthcheck; Nginx phụ thuộc app healthy.

Website: http://127.0.0.1:8080; pgAdmin: http://127.0.0.1:5050; Grafana: http://127.0.0.1:3000; Prometheus: http://127.0.0.1:9090. Loki, Promtail, database, exporter và Gunicorn không có cổng public trên máy host.

Dữ liệu được giữ bằng postgres_data, pgadmin_data, grafana_data, prometheus_data, loki_data và nginx_logs. docker compose down dừng và gỡ container/network nhưng giữ named volume. Không dùng down -v khi cần giữ dữ liệu. Sao lưu database trước khi thay đổi schema quan trọng.

Dùng docker compose logs --tail=100 app nginx để chẩn đoán ứng dụng; docker compose logs --tail=100 promtail loki khi thiếu log. Sau khi container app đổi IP, khởi động lại Nginx nếu cần để cập nhật upstream. Không chia sẻ đầu ra docker inspect chứa toàn bộ environment vì có thể lộ secret.

Role DB app và monitoring dùng mật khẩu ngẫu nhiên riêng trên máy đang chạy. Các placeholder trong .env.example là hướng dẫn, không phải mật khẩu production. Admin/tài xế demo vẫn dùng mật khẩu theo yêu cầu trước đó; cần thay bằng mật khẩu mạnh trước khi đưa lên môi trường công khai.

# 6. NGINX REVERSE PROXY VÀ SECURITY HEADERS

infra/nginx.conf có server nghe cổng 80 trong container, được map localhost:8080. location /api/ và / chuyển tiếp tới app:8000; request API truyền Host và X-Forwarded-For. File cấu hình được mount read-only, giúp cấu hình có thể review và quản lý trong Git.

Nginx áp dụng rate limit API 20 request/giây với burst 40 và trả HTTP 429 khi vượt giới hạn. client_max_body_size 3m cho ảnh tài xế; app kiểm tra giới hạn nội dung và loại ảnh riêng. server_tokens off tránh lộ chi tiết phiên bản trong response.

Security headers tại Nginx gồm X-Content-Type-Options: nosniff; X-Frame-Options: DENY; Referrer-Policy: strict-origin-when-cross-origin; Permissions-Policy chỉ cho geolocation same-origin và tắt camera/microphone. Các header dùng always, có hiệu lực cả response lỗi do Nginx trả.

App còn đặt Content-Security-Policy với script/style nội bộ và nguồn tile bản đồ cấu hình hợp lệ. Trang không dùng script CDN cho Leaflet; thư viện được lưu vendor trong repository. /metrics tại proxy public trả 404, chỉ Prometheus trong mạng Docker scrape trực tiếp app.

stub_status nghe cổng 8081 trong container, không có port mapping host. nginx-exporter kết nối http://nginx:8081/stub_status trong mạng monitoring. Như vậy dữ liệu trạng thái Nginx không được ghép vào API công khai của ứng dụng.

Bản demo đáp ứng nhánh security headers cơ bản của đề; chưa cấu hình HTTPS tự ký. Nếu triển khai public cần TLS và COOKIE_SECURE=true. Geolocation trên localhost được trình duyệt hỗ trợ; tên miền public cần HTTPS. Minh chứng header được lưu tại docs/evidence/monitoring.json.

# 7. PROMETHEUS VÀ CÁC NGUỒN METRICS

infra/prometheus.yml đặt scrape_interval 15s và năm job: taxi-api, prometheus, nginx, postgres, containers. Target lần lượt là app:8000, localhost:9090, nginx-exporter:9113, postgres-exporter:9187 và container-exporter:9108.

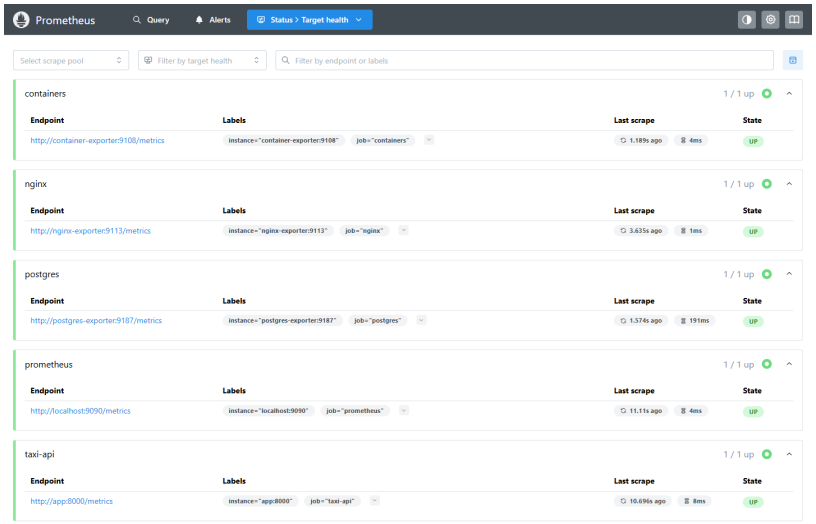
API xuất taxi_http_requests_total theo method/route/status và histogram taxi_http_request_duration_seconds. Có thể truy vấn sum(rate(taxi_http_requests_total[5m])) by (status) để quan sát throughput và histogram_quantile(0.95, sum(rate(taxi_http_request_duration_seconds_bucket[5m])) by (le)) cho p95.

nginx-exporter chuyển stub_status thành nginx_up, nginx_http_requests_total và các trạng thái connection. postgres-exporter dùng role pg_monitor, xuất pg_up và pg_stat_database_numbackends; không được dùng tài khoản quản trị dữ liệu để scrape.

Container-exporter được viết cho Docker Desktop bằng Docker Engine API GET: danh sách container theo nhãn Compose và /stats?stream=false. Xuất CPU tổng theo giây, memory working set/limit và network bytes theo service/name/id. Counter CPU được dùng rate(...[5m]) để biểu diễn số lõi CPU đang dùng.

Trong môi trường hiện tại, cAdvisor thử nghiệm không lấy được container do khác biệt Docker Desktop containerd image store. Dự án dùng metrics Docker Engine thực thay thế, không đổi nhãn hoặc giá trị giả để tạo dashboard. Thông tin này giúp giải thích lựa chọn khi demo.

Kết quả kiểm chứng: cả năm job đều up, nginx_up/pg_up/container_exporter_up bằng 1 và memory của service app có dữ liệu dương. Không chỉ kiểm tra target up: exporter có thể HTTP 200 nhưng chưa đọc được backend, nên các chỉ số backend cũng được kiểm tra.




Hình 8. Prometheus và các nguồn metrics.

# 8. GRAFANA DASHBOARD VÀ CÁCH ĐỌC

Datasource Prometheus và Loki được provision từ infra/grafana/provisioning/datasources/default.yml. Dashboard taxi-overview nằm trong infra/grafana/dashboards/taxi.json, được provision tự động khi khởi động, không phụ thuộc cấu hình thủ công trên một máy.

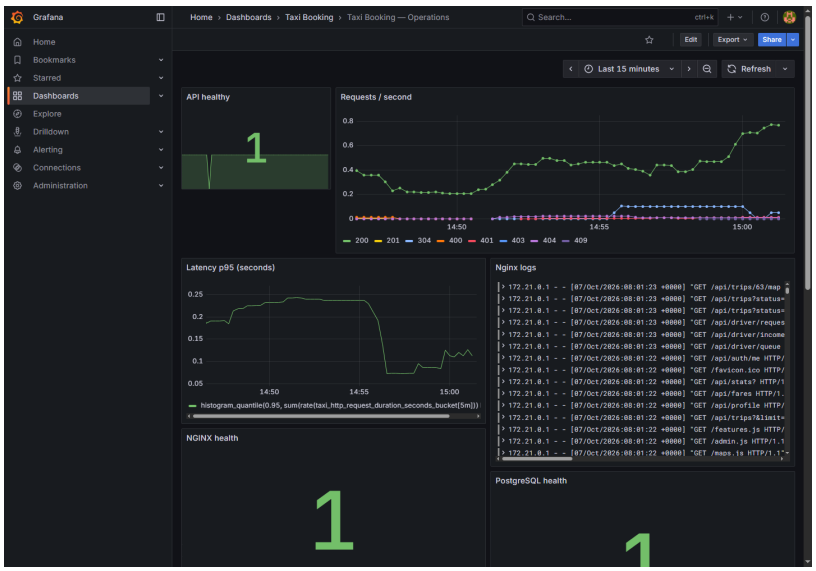
Dashboard Taxi Booking — Operations có 11 panel: API healthy, request rate, p95 latency, log Nginx, Nginx health, PostgreSQL health, scrape targets, memory container, CPU container, database connections và request rate của Nginx.

Thực hiện vài thao tác đăng nhập/đặt chuyến để tạo lưu lượng, mở dashboard với khoảng thời gian 15 phút và refresh 15 giây. Metrics CPU/rate cần ít nhất hai mẫu scrape; không kết luận lỗi khi panel rate trống ngay sau khi container mới khởi động.

Panel health phải về 1; khi container backend dừng có thể exporter vẫn up nhưng nginx_up/pg_up về 0. Throughput phản ánh số request/giây, latency dùng giây. Memory working set trừ inactive file cache; limit là giới hạn từ Docker Engine.

Grafana chỉ map cổng trên localhost, tắt tự đăng ký và dùng GRAFANA_PASSWORD. Khi trình bày có thể mở Explore để chạy PromQL hoặc LogQL. Nếu thiếu dữ liệu, kiểm tra datasource health, target Prometheus, nhãn metric và time range thay vì tự vẽ biểu đồ thay thế.

Ảnh dưới lấy từ Grafana đang chạy. Kết quả datasource health và API dashboard đã được tests/monitoring.py xác nhận. Các cấu hình JSON/YAML đều nằm trong Git để người chấm clone và chạy lại dashboard.



Hình 9. Grafana dashboard và cách đọc.

# 9. LOKI + PROMTAIL VÀ BA TRUY VẤN LOGQL

Promtail đọc /var/log/nginx/taxi-*.log từ nginx_logs, mount read-only, chạy UID/GID 65534 và bỏ capabilities. Log được gắn job="taxi-nginx", collector="promtail", gửi http://loki:3100/loki/api/v1/push. Loki lưu TSDB/filesystem và đặt retention 168 giờ.

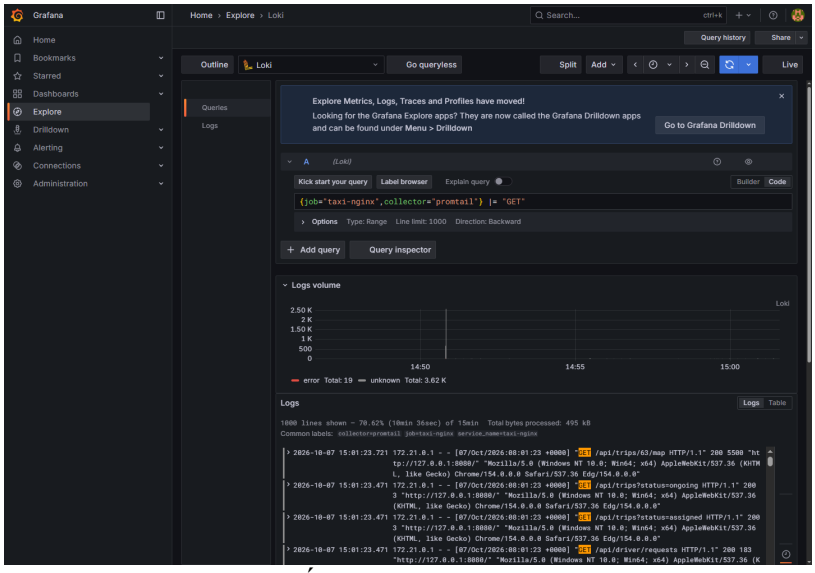
Truy vấn 1: {job="taxi-nginx",collector="promtail"}. Đây là log stream mới từ Promtail, phân biệt với dữ liệu Alloy đã lưu trước đó. Kết quả gồm timestamp và nội dung access/error log của Nginx.

Truy vấn 2: {job="taxi-nginx",collector="promtail"} |= "GET". Bộ lọc chỉ giữ dòng chứa GET, hữu ích để kiểm tra truy cập trang/API. Có thể thay chuỗi thành "/api/auth/login" hoặc " 404 " để phân tích tình huống cụ thể.

Truy vấn 3: sum(count_over_time({job="taxi-nginx",collector="promtail"}[5m])). Đây là metric query đếm tổng dòng log trong cửa sổ 5 phút. Ba truy vấn đã trả kết quả thành công bằng Loki API qua datasource proxy của Grafana.

Promtail positions lưu trên tmpfs /tmp. Sau recreate có thể đọc lại một phần file; log trùng khi restart là hạn chế demo cần nêu. Không chạy đồng thời Alloy và Promtail để tránh hai collector đẩy cùng dòng. Đề không yêu cầu log ứng dụng/database; hiện log tập trung là Nginx access/error.

Grafana xác nhận Promtail EOL từ 02/03/2026. Demo vẫn chạy Promtail theo yêu cầu học phần; phương án vận hành lâu dài là Alloy. Cấu hình Alloy nằm trong profile alloy, cần dừng Promtail trước khi bật. Tài liệu gốc: https://grafana.com/docs/loki/latest/send-data/promtail/.



Hình 10. Loki + promtail và ba truy vấn logql.

# 10. HARDENING VÀ GIỚI HẠN BẢO MẬT

App và migrate chạy user app trong image, read_only=true, tmpfs /tmp, cap_drop ALL và no-new-privileges. Đây là lớp giảm tác động khi bị khai thác; không thay thế kiểm tra xác thực/phân quyền và cập nhật dependency.

Tài khoản DB runtime taxi_app không superuser, không createdb/createrole, chỉ được DML trên bảng và dùng sequence. Kiểm thử CREATE TABLE bị SQLSTATE 42501. Migration giữ secret quản trị trong container một lần; app không nhận MIGRATION_DATABASE_URL.

Role taxi_monitor được cấp pg_monitor, tách với role ứng dụng. Database không map port host. Mạng backend và monitoring internal; chỉ app/pgAdmin/exporter cần DB mới nối backend. Giao diện web/pgAdmin/Grafana/Prometheus chỉ bind 127.0.0.1.

API dùng cookie HttpOnly/SameSite, hash scrypt, CSRF/Origin check, kiểm tra chủ sở hữu, truy vấn SQL có tham số và giới hạn input. Ảnh được kiểm tra rồi mã hóa lại bằng Pillow. Giá chuyển khoản, mã giảm giá và route quote được chốt phía máy chủ.

Ngoại lệ giám sát: container-exporter cần quyền đọc Docker socket để lấy stats, chạy root trong mạng monitoring, không publish cổng và không chuyển tiếp Docker API cho khách. Mount socket :ro không có nghĩa Docker API bị giới hạn quyền; socket vẫn là tài nguyên nhạy cảm. Các container không đặt privileged=true.

Mật khẩu DB/app-DB/monitor/pgAdmin/Grafana phải mạnh và không commit .env. Mật khẩu admin/tài xế demo 12345678 được giữ theo yêu cầu trước của người dùng; chưa đạt tiêu chí mật khẩu mạnh cho hai tài khoản này. Trước demo bảo mật/public cần đổi qua chức năng đổi mật khẩu. Báo cáo ghi đúng hạn chế, không tuyên bố toàn bộ tài khoản đều mạnh.

# 11. KIỂM THỬ VÀ MINH CHỨNG

tests/monitoring.py kiểm tra cả năm scrape target, chỉ số backend, Grafana datasource/dashboard, giao diện pgAdmin và website. Ba LogQL query đọc log của collector promtail và kiểm tra trả kết quả. Header bảo mật và /metrics public 404 cũng được xác nhận.

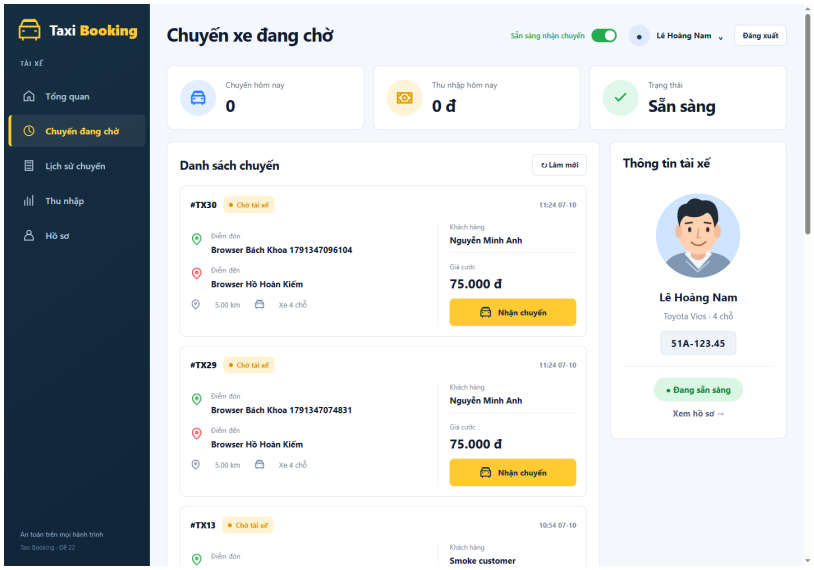
tests/hardening.py kiểm tra role runtime và thử thao tác DDL bị từ chối, Docker app non-root/read-only/capabilities, port bind localhost và network internal. File evidence chỉ giữ trường cần thiết; không lưu toàn bộ environment hoặc secret.

Các kiểm thử nghiệp vụ gồm tests/smoke.py, admin.py, features.py, password_reset.py, payment_media.py, map_features.py, promotions.py và qr_payload.py. Chức năng bản đồ/mã giảm giá có Chrome tests. Khi thay đổi hạ tầng, chạy lại kiểm thử smoke và giám sát để phát hiện lỗi kết nối/quyền DB.

Các test concurrency nhận chuyến và mã giảm giá xác nhận một tài xế/mã giới hạn một lượt không bị dùng hai lần khi request cùng thời điểm. Giá truyền giả từ client không thay được cước server. API ngoài scope trả 403/404 thay vì lộ dữ liệu người khác.

Hình minh chứng trong báo cáo là các màn hình vai trò và dịch vụ. JSON monitoring/hardening có timestamp và kết quả thực tế. Ảnh tile giả trong các test browser bản đồ được chú thích trong hướng dẫn, không sử dụng làm bằng chứng dịch vụ bản đồ public.

Bằng chứng nằm trong docs/evidence và docs/screenshots; script tạo báo cáo ở docs/build_report.py. Khi thay đổi hạ tầng hoặc thông tin sinh viên, chạy kiểm tra/thu thập ảnh và tạo lại tài liệu, tránh nộp báo cáo không khớp repository.



Hình 12. Kiểm thử và minh chứng.

# 12. QUẢN LÝ MÃ NGUỒN VÀ GITHUB

Repository Git cục bộ đã được khởi tạo với nhánh main. Các commit có ý nghĩa theo bước: commit 1 ứng dụng/PostgreSQL/pgAdmin/Nginx headers; commit 2 giám sát container/web/database bằng Prometheus + Grafana; commit 3 Loki + Promtail, LogQL, minh chứng và báo cáo.

Nội dung baseline của commit 1 bao gồm cấu hình giám sát/log đã tồn tại trước khi đối chiếu đề. Commit 2 mở rộng đủ ba nguồn metrics và hardening DB; commit 3 đổi collector mặc định thành Promtail theo tiêu chí. Báo cáo không tạo lịch sử giả về thời điểm đã làm từng chức năng.

Tác giả commit tạm thời là Taxi Booking Project vì chưa có họ tên/email sinh viên. Tạo tài khoản GitHub và tên repository theo mã số sinh viên, dùng email tài khoản sinh viên cho các commit tiếp theo. Cần bổ sung thông tin này trước khi nộp.

Sau khi sinh viên tạo repo trống và cung cấp URL: git remote add origin https://github.com/TEN_TAI_KHOAN/MSSV-taxi-booking.git; git push -u origin main. Xác thực bằng Git Credential Manager/PAT theo giao diện GitHub; không dán token vào README hoặc file cấu hình.

Trước push: git status --short; git log --oneline; git ls-files .env phải không có kết quả. .env.example được commit nhưng chỉ chứa placeholder. README có hướng dẫn Docker, URL dịch vụ, tài khoản, phân quyền, backup, metrics, LogQL, hardening và link báo cáo.

Trạng thái GitHub hiện tại: chờ URL repository và xác thực của sinh viên. Không có bằng chứng remote/push trong báo cáo vì thao tác này chưa thực hiện. Tiêu chí GitHub chỉ hoàn tất khi repo truy cập được, có lịch sử commit và README theo đúng tài khoản/mã số sinh viên.

# 13. KỊCH BẢN DEMO, KẾT LUẬN VÀ TÀI LIỆU

Demo 1: docker compose ps -a, chỉ ra app/db healthy và migrate Exited (0). Mở website qua Nginx, đăng nhập khách đặt chuyến xe máy/ô tô với mã WELCOME20, đăng nhập tài xế nhận/bắt đầu/hoàn thành và admin xem lịch sử/giá sau giảm.

Demo 2: mở pgAdmin, kết nối db và Query Tool xem dữ liệu trips/users/vehicles. Mở Prometheus /targets cho năm job UP, truy vấn pg_up và memory service app. Mở Grafana dashboard để chỉ rõ panel web, database và container.

Demo 3: tạo thêm request website, mở Grafana Explore datasource Loki rồi chạy ba query trong mục 9. Giải thích log path, volume chung, label collector và luồng Promtail → Loki → Grafana. Cho thấy cấu hình nằm trong repository.

Demo 4: chạy tests/hardening.py và giải thích non-root, read-only, internal network, không mở DB port, role DML, Nginx headers. Nêu ngoại lệ Docker socket của exporter và mật khẩu demo còn yếu. Không công khai secret trên màn hình khi trình bày.

Kết quả: hệ thống Docker Compose chạy ứng dụng, DB tool, reverse proxy, monitoring và log tập trung; có kiểm thử và báo cáo minh chứng. Phần cần sinh viên hoàn thiện trước nộp là thông tin bìa, tài khoản/repo GitHub theo MSSV và đổi mật khẩu demo nếu chấm tiêu chí mật khẩu mạnh.

Nguồn chính thức: Docker Engine API https://docs.docker.com/reference/api/engine/; Nginx exporter https://github.com/nginx/nginx-prometheus-exporter; PostgreSQL exporter https://github.com/prometheus-community/postgres_exporter; Prometheus https://prometheus.io/docs/; Grafana/Loki https://grafana.com/docs/loki/latest/; chính sách tile https://operations.osmfoundation.org/policies/tiles/. Cấu hình và kiểm chứng của dự án là nguồn chính cho kết quả thực nghiệm.